Organizing Sidekiq Job Code

Other than keeping the code in your jobs to a minimum (as discussed in *Create a New Sidekiq Job*), the main issue with organizing Sidekiq job code is managing duplication. There are three ways to do so: using middleware, using generators, and using a common base class. Using a base class should be familiar to you, since that is how most code is shared in Rails codebase. One problem with our Sidekiq jobs is that `ApplicationJob` is using Active Job.

No Need for Active Job

Active Job is a generalized interface for queuing jobs,[24] that can use any job library underneath, be it Sidekiq, Resque, or something else. This is useful when an external library or Rails' internals need to queue a job. This is less useful when writing a Rails application.

Active Job doesn't alleviate the need to understand Sidekiq in detail—you still need to know how Sidekiq manages parameters, Redis, and errors. If you use Active Job, you have more stuff to to learn, without any real benefit to doing so.

Active Job also modifies the parameters queued with a job, namely if you pass in an Active Record. It will handle putting the ID into the job payload, and then using `find` to look it up before the job executes. But it only works with Active Records, and it only saves a single line of very obvious, very hard-to-mess-up code. It's not worth it.

To codify the decision *not* to use Active Job, let's change `ApplicationJob` to be a base class for Sidekiq jobs. A comment documents what you've done and you can include the `Sidekiq::Job` module, so that jobs don't have to include that themselves.

snapshots/4-2/sidekiq-book/app/jobs/application_job.rb

```
# All jobs use Sidekiq, not Active Job
```